StockSync

Product Batch Traceability and Recall Management System

DBMS Level 3 Project Report

Team Member	SRN
Rekha Dhorigol	PES1UG24CS370
Reet Porwal	PES1UG24CS369

Section: G
Repository: https://github.com/reetporwal66/inventory-traceability

Project overview

StockSync is an inventory traceability system for managing products, suppliers, batches, warehouses, stock levels, reservations, transfers, shipments, quality defects and product recalls. MySQL stores transactional records and enforces relational integrity. MongoDB stores operational event documents synchronized through a MySQL outbox workflow.

1. Abstract

StockSync is an inventory traceability and recall management system that supports product and batch management, supplier associations, warehouse inventory, reservations, transfers, shipments, quality defect reporting and product recall recovery. The application uses a web interface and MySQL for transactional data, while MongoDB stores operational event documents synchronized through a transactional outbox workflow.

The system uses a relational database design with primary keys, foreign keys, constraints, indexes, stored procedures, views and triggers to maintain data integrity and support operational reporting. Product management enables authorized users to register products with unique SKUs and relevant classification and stock-reorder information. Batch management supports the creation of traceable batches associated with existing products and suppliers, including batch-specific details. Product creation also records a corresponding event in the outbox within the same database transaction. These features extend the system's foundation for inventory control, batch-level traceability and reliable event synchronization.

2. Problem Statement and Objectives

Inventory operations require reliable stock counts, traceability between batches and destinations, and timely handling of shipment discrepancies, defects and recalls. Scattered or inconsistent records can make it difficult to identify affected stock and recipients. StockSync aims to provide a structured data model and operational workflow for inventory control and traceability.

Objectives

- Maintain product, supplier, batch, warehouse and inventory records.
- Enable authorized users to create and maintain product master records with validated fields and unique product identifiers.
- Support batch creation linked to existing products and suppliers, maintaining batch-level traceability.
- Support stock reservation, cancellation and inter-warehouse transfer.
- Track shipments, shipment items, recipients and delivery discrepancies.
- Record quality defects, recalls, affected batches and recovery actions.
- Provide reporting views for inventory status, low stock, recall tracing and shipment discrepancies.
- Publish selected inventory events to MongoDB through an outbox mechanism.

3. Functional Requirements

Area	Requirement
Inventory	Maintain inventory quantities and support stock operations across warehouses and batches.
Reservations	Reserve available stock and cancel reservations while keeping reserved quantities consistent.
Transfers	Transfer stock between locations with transactional validation.
Shipments	Manage shipment items and enforce valid shipment-status transitions.
Quality and recalls	Record defects, quarantine stock, identify affected batches and record recovery.
Reporting	Query inventory overview, low-stock items, recall recipients and shipment discrepancies.
Event synchronization	Record selected inventory changes in an outbox and synchronize events to MongoDB.
Product management	Create and list product master records with unique SKUs, product names, categories, units and reorder-level information.
Batch management	Create and list batches associated with existing products and active suppliers, with validation of batch details.

4. Technology and Database Design

The application uses a web interface and server-side application logic, with MySQL for transactional relational data and MongoDB for event-oriented documents. The Next.js application provides product and batch management interfaces backed by server-side API routes. The product API supports listing and creating product records, while the batch API supports batch creation and listing. Database transactions are used for operations that involve multiple related writes, and validation is applied to incoming data. Product creation records a PRODUCT_CREATED event in the MySQL outbox within the same transaction. Supporting application libraries include mysql2 and mongodb, with Zod and authentication-related libraries used elsewhere in the application.

5. Entity-Relationship Design

The ER diagram summarizes the principal entities and their relationships. Products are associated with batches and suppliers; inventory associates stock with warehouses and batches; shipment records connect shipments with recipients and shipment items; quality and recall entities support defect handling, affected-batch tracing and recovery.

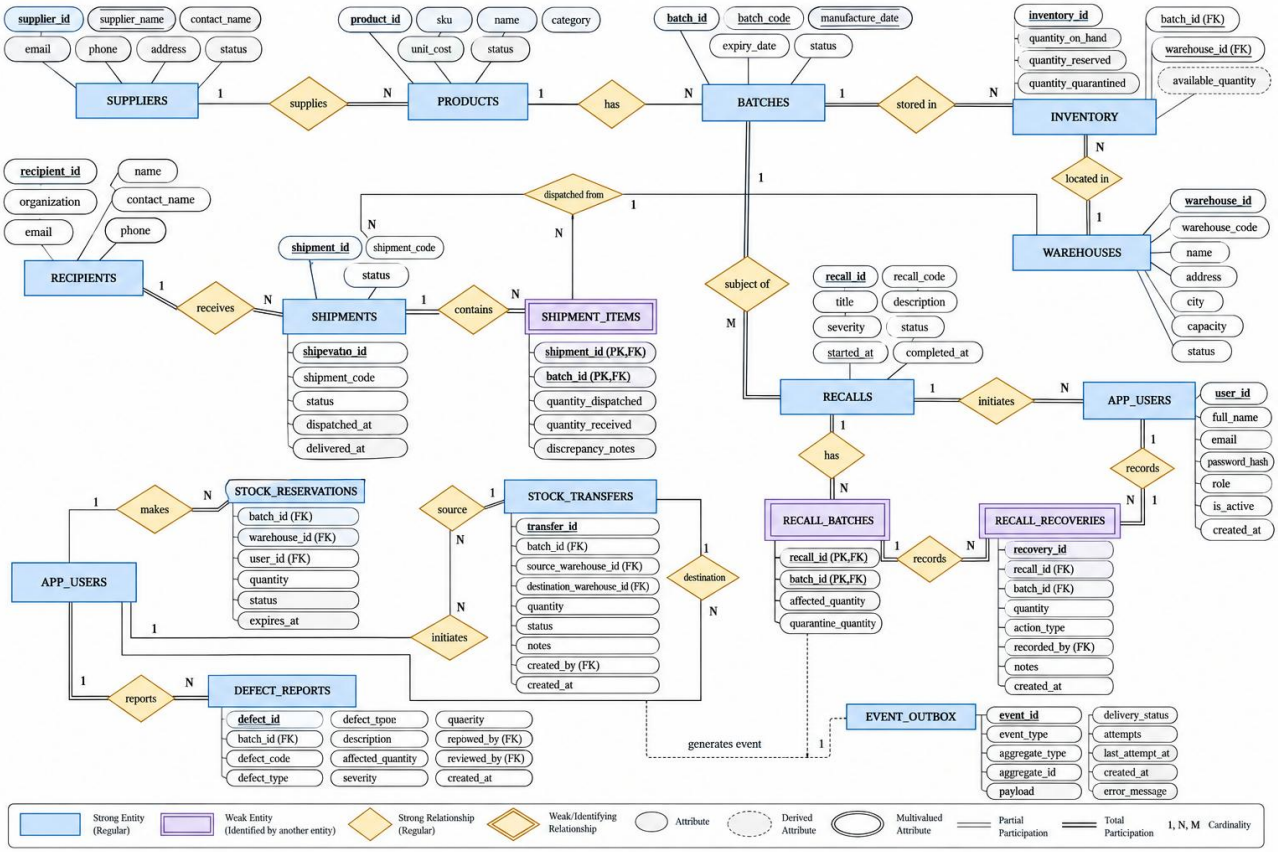
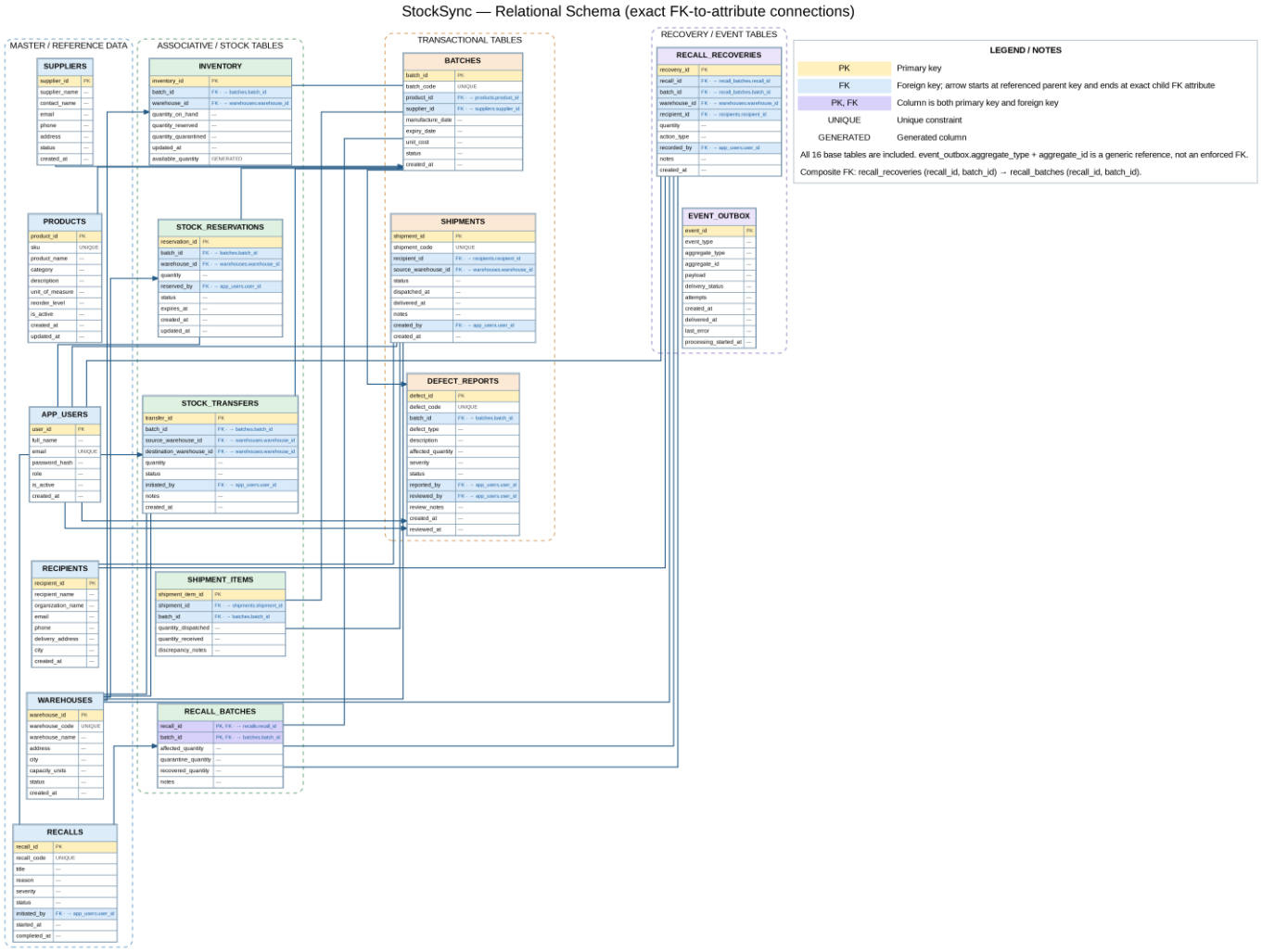


Figure 1. Entity-Relationship Diagram

6. Relational Schema

The relational schema maps the conceptual model to tables, primary keys and foreign-key relationships. It supports referential integrity across inventory operations, shipment traceability, quality reporting and recall recovery.

(Diagram on next page)



For better clarity: (next page)

StockSync — Relational Schema

PK = primary key (underlined); FK = foreign key; UK = unique key; derived = generated attribute.
Relation attributes follow the schema implemented in MySQL. Composite keys are shown explicitly.

APP_USERS

user_id	PK
full_name	
email	UNIQUE
password_hash	
role	
is_active	
created_at	

PRODUCTS

product_id	PK
sku	UNIQUE
product_name	
category	
description	
unit_of_measure	
reorder_level	
is_active	
created_at	
updated_at	

SUPPLIERS

supplier_id	PK
supplier_name	
contact_name	
email	
phone	
address	
status	
created_at	

BATCHES

batch_id	PK
batch_code	UNIQUE
product_id	FK→PRODUCTS
supplier_id	FK→SUPPLIERS
manufacture_date	
expiry_date	
unit_cost	
status	
created_at	

WAREHOUSES

warehouse_id	PK
warehouse_code	UNIQUE
warehouse_name	
address	
city	
capacity_units	
status	
created_at	

INVENTORY

inventory_id	PK
batch_id	FK→BATCHES
warehouse_id	FK→WAREHOUSES
quantity_on_hand	
quantity_reserved	
quantity_quarantined	
available_quantity	DERIVED
updated_at	
(batch_id, warehouse_id)	UNIQUE

STOCK_RESERVATIONS

reservation_id	PK
batch_id	FK→BATCHES
warehouse_id	FK→WAREHOUSES
quantity	
reserved_by	FK→APP_USERS
status	
expires_at	
created_at	
updated_at	

STOCK_TRANSFERS

transfer_id	PK
batch_id	FK→BATCHES
source_warehouse_id	FK→WAREHOUSES
destination_warehouse_id	FK→WAREHOUSES
quantity	
status	
initiated_by	FK→APP_USERS
notes	
created_at	

RECIPIENTS

recipient_id	PK
recipient_name	
organization_name	
email	
phone	
delivery_address	
city	
created_at	

SHIPMENTS

shipment_id	PK
shipment_code	UNIQUE
recipient_id	FK→RECIPIENTS
source_warehouse_id	FK→WAREHOUSES
status	
dispatched_at	
delivered_at	
notes	
created_by	FK→APP_USERS
created_at	

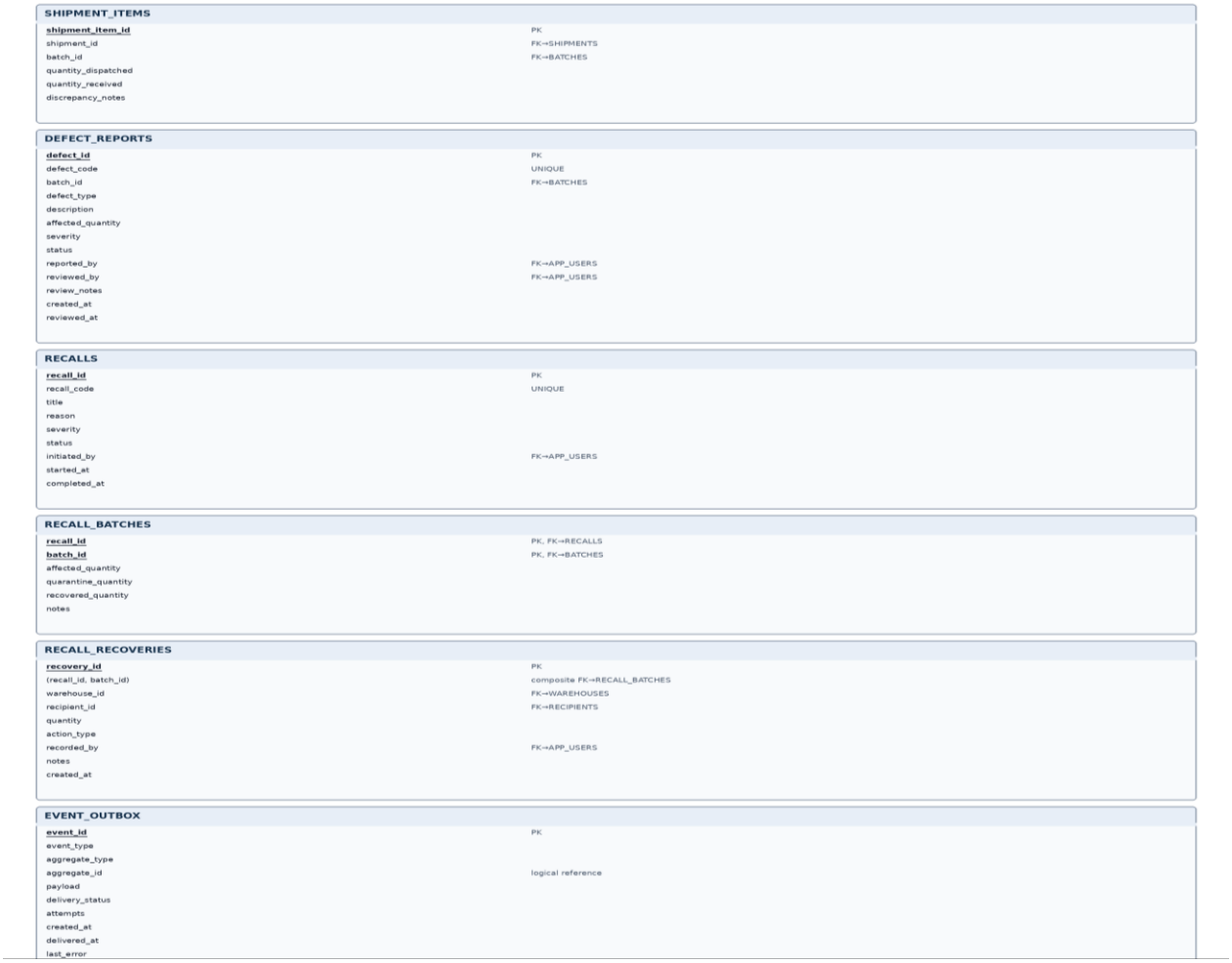


Figure 2. Relational Schema

7. Data Dictionary Summary

Table	Purpose / key attributes
app_users	Application users and role-related information.
products	Product master records, unique SKU, product name, category, unit, reorder level and product status.
suppliers	Supplier information associated with sourced products/batches.
batches	Batch-level traceability, batch identifier, product association, supplier association and batch-specific details.
warehouses	Warehouse/location master records.
inventory	Stock quantities for batches at warehouses.
stock_reservations	Reservations of inventory and their lifecycle status.
stock_transfers	Records of stock movement between locations.
recipients	Shipment recipient details.
shipments	Shipment status and shipment-level information.
shipment_items	Products/batches and quantities associated with shipments.
defect_reports	Defect codes, details, severity/status and related batch.
recalls	Recall records and their status/details.
recall_batches	Association between recalls and affected batches.
recall_recoveries	Recorded recovery actions and outcomes.
event_outbox	Events queued for downstream delivery and delivery status.

8. Integrity, Indexing and Advanced SQL

Primary keys identify records, while foreign keys connect related entities and help prevent orphaned records. Unique constraints protect business identifiers where defined. Transactional procedures centralize operations that must update related stock records consistently.

Stored procedures

• **sp_reserve_stock** — reserves stock according to the procedure's validation and update logic.

• **sp_cancel_reservation** — cancels a reservation and updates related reservation quantities/status.

• **sp_record_quarantine** — records stock quarantine as part of quality and recall handling.

• **sp_transfer_stock** — handles stock transfer between locations.

Reporting views

• **vw_inventory_overview** — summarizes inventory information.

• **vw_low_stock** — identifies items requiring low-stock attention.

• **vw_recall_recipient_trace** — supports tracing recall-related shipments and recipients.

• **vw_shipment_discrepancies** — exposes shipment discrepancy information.

Triggers and outbox

The triggers **trg_inventory_insert_outbox** and **trg_inventory_update_outbox** add event records to the outbox for selected inventory insert/update operations. The outbox supports reliable separation between MySQL transactions and downstream event delivery.

Application-level transactions and validation

Product creation is implemented through a server-side API that validates required fields and checks user authorization. The operation inserts the product record and its PRODUCT_CREATED event into the event_outbox table within a MySQL transaction. If the operation fails, the transaction is rolled back. Duplicate SKU values are handled as conflicts rather than being accepted as new products.

Batch creation validates batch details and checks the associated product and supplier before inserting the batch record. The operation uses a database transaction and records a BATCH_CREATED event in the outbox. Duplicate batch codes and invalid input are handled by the API's validation and error-handling logic.

9. MongoDB Event Model

MongoDB is used for event-oriented records in database **inventory_events**, collection **events**. The synchronization workflow reads pending outbox events, writes event documents to MongoDB and updates delivery status. This design keeps MySQL as the source of truth for transactional inventory data while enabling flexible event storage and downstream inspection.

10. Testing and Evidence

The project includes evidence for the main database and application workflows. The following checks describe the expected behavior covered by the project test scenarios.

Test area	Expected result
Build and lint	Application linting and production build complete without errors.
Reservation lifecycle	Creating and cancelling a reservation restores the reserved quantity to the expected value.
Shipment status	Valid status transitions succeed; invalid transitions are rejected with an appropriate conflict response.
Recall and quarantine	Quarantine and recall actions create the corresponding traceability and recovery records.
MySQL-to-MongoDB synchronization	Pending outbox events are written to MongoDB and their delivery status is updated.
Product creation	A valid product is saved and appears in the product listing.
Duplicate SKU validation	A duplicate SKU is rejected with a conflict response.
Batch creation	A valid batch is saved with its product and supplier associations.
Batch validation	Invalid batch details or invalid product/supplier associations are rejected.
Product event recording	Product creation and its outbox event are committed together; a failed transaction does not leave a partial product creation.
Static checks	ESLint and the production build complete successfully.

11. Setup and Operational Considerations

Use the project README and .env.example to configure the application and its database connections. Set environment variables for the local environment rather than committing secrets. Import the SQL setup into a dedicated development database and confirm the database user has the privileges required by the setup and application.

The SQL setup script contains destructive table-reset statements and sample-data operations. It should be run only against an intended development/test database after reviewing the target connection. The script also changes foreign-key-check behavior during import; verify the resulting schema and data integrity after setup. Definer clauses may require adjustment for a local MySQL account.

Project evidence

Screenshots documenting the user interface, MySQL queries and MongoDB event records are maintained separately in the project's **docs/screenshots/** directory. This report includes only the ER diagram and relational schema as figures.

12. Conclusion

StockSync integrates product and batch management with relational inventory control and operational traceability for reservations, transfers, shipments, quality defects and recalls. The product and batch APIs extend the application by supporting product registration, batch creation and associations between products and suppliers. MySQL transactions, constraints and validation help maintain consistency across related records, while the transactional outbox supports synchronization of selected operational events to MongoDB. Stored procedures and reporting views provide support for inventory operations and traceability queries. The system therefore provides a structured foundation for managing inventory records, tracking batches and supporting recall-related workflows.